

# AADSP1 –Skripta

## I UVOD U LINUX U NAMENSKIM SISTEMIMA

### 1. Prednosti Linuks otvorenog koda u namenskim sistemima:

- Mogućnost ponovnog korišćenja komponenti
- Otvoreni kod nudi mnoge prednosti, brži razvoj, ne moraju da se razvijaju stvari koje su već jednom razvijene
- mala cena, besplatan softver može da se ugradi na željenom broju uređaja besplatno
- čak i alati za razvoj mogu biti besplatni
- dozvoljava punu kontrolu nad softverom u vašem sistemu
- zbog šire primene, na milionima sistema imaju dosta velik kvalitet
- olakšava testiranje novih funkcionalnosti, jednostavan pregled novih mogućnosti i rešenja
- podrška u zajednici omogućava ubrzavanje rešavanja problema kada razvijate vaš sistem

### 2. Primeri namenskih sistema na kojima se nalazi Linux:

- Personalni ruteri
- Televizori
- POS terminali
- Laserska mašina za sečenje
- Mašine za vinogradarstvo

### 3. Procesori i arhitekture na kojima se koristi Linux:

- x86 i x86-64
- ARM(multimedijalna i industrijska primena)
- PowerPC(u realnom vremenu,industrijska primena)
- MIPS (mrežna primena)
- SuperH(set top box i multimedijalna primena)
- Blackfin(DSP arhitekture)
- MicroBlaze(Xilinx FPGA softver)
- osim alata za prevodjenje,bootloader-a i kernela sve ostale komponente su generalno nezavisne od arhitekture

### 4. Preko kojih komunikacionih magistrala može da komunicira Linux kernel:

- I2C
- SPI
- CAN
- 1-wire
- SDIO
- USB

## 5. Tipovi hardverskih platformi

- platforme za **EVALUACIJU** (skupe, razne periferije, nisu za krajnji proizvod)
- komponente u vidu **MODULA** (pločice koje imaju samo osnovne komponente, može se koristiti za krajnje proizvode u malim ili srednjim serijama)
- platforme za **RAZVOJ U ZAJEDNICI** (lako dostupno, dobra podrška, niska cena, manje periferija od platformi za evaluaciju, mogu se koristiti za krajnji proizvod)
- platforme po **MERI** (najbolje za krajnji proizvod)

## II ARHITEKTURE ZA NAMENSKE LINUX SISTEME

### 6. Programske komponente:

- alat za **prevodjenje za drugu platformu** (Cross-compilation)
- **BOOTLOADER** (odgovoran za izvršenje kernela)
- **KERNEL** (sadrži proces menadžment i memorijski menadžment, mrežni stek, rukovaoce uređajima i nudi servise za aplikacije u korisničkom prostoru)
- **C biblioteka** (sprega između kernela i aplikacija u korisničkom prostoru)
- **BIBLIOTEKE i aplikacije** (dobijene od drugih ili napravljene „Kod kuće“)

### 7. Šta je potrebno za razvoj Linuxa za određeni proizvod?

- razvoj **Board Support Package** (sadrži bootloader i kernel sa odgovarajućim rukovaoima za ciljani hardver)
- **integracija** sistema (sklapanje bootladera, kernela i ostalih delova sistema u celinu)
- **razvoj** aplikacija (koriste posebno odabrane biblioteke)

## III RAZVOJNO OKRUŽENJE LINUX-a U NAMENSKIM SISTEMIMA

### 8. Šta je razvojna, a šta odredišna platforma?

- **HOST** (razvojne radne stanice, obično moćan PC)
- **TARGET** (namenski sistem koji se razvija)
- Povezani su uvek serijskom vezom u svrhu debugovanja, često Ethernet vezom, ponekad JTAG spregom za debugovanje na niskom nivou

### 9. Program za komunikacija preko serijske linije?

- Minicom, Picocom, Gtterm, Putty, itd.
- na kursu se koristi **Picocom** (sudo apt-get install picocom)
- pokreće se komandom ( picocom -b BAUD\_RATE /dev/SERAIL\_DEVICE)
- prekida se komandom (CTRL-A, CTRL-X)
- SERAIL\_DEVICE je tipično ttyUSBx (USB na serijsku vezu), ttySx (fizički serijski portovi)

## IV ALATI ZA UNAKRSNO PREVOĐENJE- DEFINICIJA I KOMPONENTE

### 10. Definicija alata za unakrsno i izvorno prevođenje (CROSS-compiler and nativ compiler)

- alati za **unakrsno** prevođenje se pokreću na radnoj stanici i generišu kod za odredišnu platformu.
- dok se **ugrađeni** alati za prevođenje pokreću ba radnih stanici i generišu kod za radnu stanicu.

### 11. Koje su tri vrste mašina u bild procedurama?

- **bild** mašina (pravi se alat za unakrsno prevođenje)
- **host** mašina (završava se alat za unakrsno prevođenje i pravi binarnu, izvršnu datoteku za odredišnu platformu)
- **target** mašina (gde se izvršavaju binarne datoteke napravljene od strane alata za unakrsno prevođenje)

### 12. Koje bild procedure alata za prevođenje postoje?

- **Nativ build** (sve se pravi i izvršava na jednoj arhitekturi, odnosno na radnoj stanici)
- **Cross build** (alat se pravi i prevodi na radnoj stanici, a izvršava se na odredišnoj arhitekturi, najčešće korištena procedura)
- **Cross-native build** (alat se pravi na radnoj stanici, a pravi i izvršava izvršnu, binarnu datoteku, na odredišnoj arhitekturi)
- **Canadian build** (alat se pravi na prvoj arhitekturi, izvršna, binarna datoteka se pravi na drugoj arhitekturi, dok se izvršavaju izvršna, binarna datoteka na trećoj arhitekturi)

### 13. Šta je i čemu služi Binutils?

- skup alata za generisanje i manipulisanje binarnim datotekama za zadatu CPU arhitekturu
- as, assembler, generiše binarnu datoteku na osnovu asemblerskog izvornog koda
- ld, povezič
- ar, ranlib, za generisanje .a arhiva, korišćenih za biblioteke
- objdump, readelf, size, nm, strings, za inspekciju binarnih datoteka. Koristan alat za analizu!
- strips, odbacuje deo binarnih datoteka koje se koriste samo za debugovanje (smanjuje veličinu)

### 14. Šta su i čemu služe kernel zaglavlja?

- C biblioteka i prevedeni program zahtevaju interakciju sa kernelom, iz njih uzimaju sistemske pozive i njihove brojeve, definicije konstanti, strukture podataka
- <linux/...> i <asm/...> na tim lokacijama su dostupna zaglavlja kernel
- starije verzije kernel zaglavlja će na novijem kernelu raditi bez problema, ali neće moći da koriste nove sistemske pozive, strukture podataka, itd.
- binarne datoteke generisane alatom novijih kernel zaglavlja bi radile na starijem kernelu, ako nisu korištene funkcionalnosti novijeg kernel.
- Makefile headers\_install- raspakivanje iz kernel izvornog koda

### 15. GCC?

- GNU Compiler Collection
- prevodi razne programske jezike i generiše kod za razne arhitekture

## 16. C biblioteka?

- sprega između aplikacija i kernela
- olakšava razvoj aplikacija
- glibc, uClibc, musl, dietlibc, newlib, itd.

## V OPCIJE ALATA ZA PREVOĐENJE

### 17. Šta je ABI(Application Binary Interface)?

- alat za prevođenje koji se koristi za generisanje binarnih datoteka, mora biti definisan
- definiše pozivne konvencije (argumenti, povratne vrednosti, sistemski pozivi) i organizacije struktura (poravnanja, itd.)
- sve binarne datoteke u sistemu moraju biti prevedene korišćenjem istog ABI-ja, i kernel mora da ume da rukuje istim ABI-jem
- ARM ima dva glavna (OABI i EABI)
- MIPS koristi nekoliko (o32, o64, n32, n64)

## VI SEVKENCA POKRETANJA (BOOT)

### 18. Šta je Bootloader?

- vrši osnovnu inicijalizaciju hardvera, učitava binarne datoteke aplikacije, najčešće kernela operativnog sistema, iz fleš memorije, sa mreže ili sa drugog tipa memorije.
- može da uključi i dekompresuje binarne datoteke aplikacije.
- izvršava aplikacije.

### 19. Bootloader na BIOS-baziranoj x86 arhitekturi?

- BIOS je odgovoran za inicijalizaciju osnovnog hardvera i učitavanje veoma malog dela koda iz trajne memorije.
- ovaj deo koda je tipično bootloader prve faze koji će učitati potpun bootloader
- GRUB (najmoćniji bootloader)
- Syslinux (za bootovanje preko mreže ili prenosnog medijuma USB, CD-ROM)

### 20. Koliko ima slučajeva Boot-ovanja na namenskim procesorima?

- **slučaj 1** kada se uključi CPU počne da izvršava kod na fiksnoj adresi, hardver mora biti dizajniran tako da NOR flash cip bude povezan tako da je dostupan na adresi na kojoj CPU počinje izvršenje instrukcija, NOR omogućava slučajan pristup, nepraktičan zahteva NOR FLASH
- **slučaj 2** CPU ima integrisan kod za boot-ovanje u ROM-u, detalji zavise od CPU, kod za boot-ovanje pokreće bootloader prve faze iz skladišta u SRAM, bootloader prve faze je ograničene veličine zbog SRAM-a, iz DRAM-a učitava bootloader druge faze u RAM i ostale hardverske uređaje

### 21. Šta je TFTP?

- Mrežni prenos od razvojne stanice do U-boot-a na ciljanoj platformi se odvija kroz TFTP (Trivial File Transfer Protocol) donekle sličan FTP, ali bez autentifikacije preko UDP-a
- TFTP server je potreban na razvojnoj stanici (sudo apt-get install tftpd-hpa) sve datoteke u

/var/lib/tftpboot su vidljive kroz TFTP

-TFTP klijent je integrisan u U-BOOT konfiguracija ipaaddr i serverip promenljive okruženja

## VII BASH SKRIPTA

### 22. Odlike BASH skripta?

- skripta može biti izgrađena u delovima veličine sekcija
- postoji samo (relativno) mali skup Shell – specifičnih operatora i opcija da se nauče
- sintaksa je jednostavna
- „Zanat skriptinga“ nije teško savladati

### 23. Zašto koristiti BASH?

- Shell skript je brz i pljav (quick and dirty) metod kreiranja protipa složene aplikacije
- Dobijanje čak i ograničenog podskupa funkcionalnosti za rad u scenariju je često korisna prva faza u razvoju projekta
- Na ovaj način, struktura aplikacije može biti testirana i menjana
- Glavne zamke mogu biti pronađene pre nastavka kodiranja

### 24. Kada NE koristiti BASH?

- Resource-intenzivni zadaci, naročito kada je brzina bitan faktor (sortiranje, heširanje, rekurzije)
- Postupci koji se odnose na teške uslove rada matematičkih operacija, posebno aritmetika u pokretnom zarezu, proizvoljne precizne kalkulacije, ili kompleksni brojevi
- Kada je potrebna cross-platform portabilnost
- kompleksne aplikacije, gde je struktuirano programiranje neophodno (provera tipa promenljivih, funkcija prototipa, itd)
- kritične aplikacije od kojih zavisi budućnost (npr. Kompanije)
- Situacije u kojima je važna bezbednost, gde bi trebalo da se garantuje integritet vašeg sistema i štiti od upada, pucanja i vandalizma
- Ekstenzivan rad sa datotekama
- Potrebna ugrađena podrška za višedimenzionalne nizove
- Treba da se generiše / manipuliše grafikom ili sa GUI

## IIIX MAKE ALAT I MAKEFILE

### 25. Objasni proces prevođenja?

- Prevođenje programa koji se sastoji od velikog broja izvornih datoteka i zaglavlja može da bude komplikovano i da oduzme mnogo vremena
- Kucanje dugačkih komandi za prevodjenje koda je sklono greškama.
- Posle bilo kakve promene potrebno je prevesti fajl koji je promenjen

### 26. Make alat?

- Obično su ulazi projekta izvorne datoteke, a izlaz je jedna ili više izvršnih datoteka ili bibliotek.
- make je alat koji se koristi za efikasno prevođenje programa
- Automatski detktuje delove programa koji treba da se prevedu i izdaje komande da se prevođenje

izvrši ( prevodi program ponovo ako izlaz ne postoji ili je došlo do promene u kodu)

- make čita instrukcije iz txt datoteke
- podrazumevano ime datoteke je makefile ili Makefile
- opcijom -f možemo iskoristiti bilo koju datoteku za ulazne instrukcije (make -f InstrukcijaZaPrev...)
- opšti oblik sintakse make (make [-f makefile] [opcije]... [ciljevi]...)
- razni debug ispisi u toku prevođenja se mogu uključiti opcijom -d

## 27. Od čega se sastoji makefile?

- **eksplicitnih pravila** (opisuju kada i kako da se preved jedna ili više datotek)
- **implicitnih pravila** (opisuje kada i kako da se preved grupa datoteka na osnovu njihovih imena)
- **definicija promenljivih** (definiše tekstualne promenljive koje će kasnije biti zamenjene)
- **direktiva** (naređuju izvršenje akcija dok se analizira makefile)
- \*uključivanje drugog makefile-a                      \*uslovno izvršenje određenog dela
- \*definicija promenljive u više linija
- **komentara** – počinju sa # i traju do kraja linije

## 28. Kako se pravi objektna datoteka?

- zavisnosti su izvorne datoteke (i zaglavlja) potrebne za pravljenje objektna datoteke, u slučaju bilo kakve promene u zavisnostima, opet mora da se prevede

- primer:

```
app.o: app.c app.h math.h factorial.h libwrite.h
      gcc -O2 -c -I../inc -I../libinc ../src/app.c -o app.o
```

## 29. Kako se pravi izvršna datoteka?

- zavisnosti su sve objektna datoteke potrebne za pravljenje izvršne datoteke u slučaju bilo kakve promene bilo koje ulazne datoteke promeniće se i izvršna datoteka

-primer:

```
app: app.o factorial.o math.o
      gcc -L../lib -lwrite app.o factorial.o math.o -o app
```

## 30. Promenljive u Makefilu?

- prilikom izvršenja pravila promenljive se zamenjuju sa njihovim vrednostima
- čine kod čitljivijim i lakšim za konfigurisanje

CC = gcc

app.o: app.c app.h ...

\${CC} -O2 -c -I...

- promena prevodioca se ovde radi tako što se zameni samo vrednost promenljive
- Vrednost promenljive se može postaviti:
  - \*iz komandne linije (make all CC= /usr/bin/gcc)
  - \* preko promenljive okruženja (export CC= /usr/bin/gcc)

## 31. Korišćenje specijalnih karaktera u make fajlovima?

- Vajld-karte su:

\*: menja bilo koji string od 0 ili više karaktera (rm -f \*.o – briše sve objektno datoteke)

?: menja jedan karakter (rm -f app.? – briše sve datoteke sa nazivom app)

[lista\_karaktera]:menja jedan karakter sa nekim iz liste(rm -f app[1-3].o – briše sve datoteke ako postoje app1, app2, app3)

-ukoliko je potrebno da se u imenu iskoristi karakter koji je l valid-karta pre njega se piše \

### 32. Implicitna pravila makefila?

- da bi se koristilo implicitno pravilo potrebno je napisati pravilo bez komande ili ne napisati pravilo uopšte (app: app.o math.o factorial.o)

\$(CC) -o app app.o math.o factorial.o

- Lista pravila zavisi od operativnog sistema

- Prevođenje C programa (CC), prevođenje asemblerkih programa (AS), prevođenje objektnih datoteka (LD)

- moguće je zabraniti korišćenje implicitnih pravila (-r parameter -no-nuiltin-rules)

- pravila koriste predefinisane promenljive

### 33. Automatske promenljive makefila?

-\$@ (naziv datoteke iz odredišta)

-\$% (naziv člana odredišta, kada je odredište član arhive)

-\$< (naziv prve zavisnosti)

-\$? (naziv svih zavisnosti novijih od odredišta)

-\$^ (naziv svih zavisnosti razdvojenih razmacima bez duplikata)

-\$+ (slično kao \$^ s tim što se duplikati pojavljuju u redosledu u kom su navedeni)

## IX KERNEL

### 34. Glavne odlike Linuks Kernela?

- Portabilnost i podrška za fizičke arhitekture

- Skalabilnost (može da se pokrene od najslabijeg hardvera do super računara)

- Podržava standard i interoperabilnost

- Izdašna podrška za rad sa mrežama

- Sigurnost (kod pregleda veliki broj stručnjaka, teško može doći do greške)

- Stabilnost i pouzdanost

- Modularnost (može da uključi potrebnu funkcionalnost u toku rada)

- Lak za programiranje (može da se uključi iz postojećeg koda, postoji puno kursin na internetu)

### 35. U kom programskom jeziku je implementiran i kako se prevodi Kernel?

- Linuks kernel je pretežno implementiran u C programskom jeziku.

- Prevodi se sa GNU C, ne može da se koristi bilo koji ANSI C prevodilac

- neke GNU C ekstenzije koje se koriste u kernelu su inline C, inline assembler

### 36. Na koji način se kodira u kernelu?

- Nikad se ne koriste brojevi sa pokretnim zarezom, kod može biti pokrenut na procesoru koji nema podršku za brojeve sa pokretnim zarezom, brojevi sa pokretnim zarezom se mogu simulirati u kernelu
- Svi simboli se definišu kao statički, osim onih koji se eksportuju

### 37. Arhitektura kernela?

- Korisnički prostor (aplikacija1, aplikacija2, C biblioteka)
- Kernel Prostor (Sprema za sistemske pozive komuniciranja sa C bibliotekom, upravljanje procesima, memorijom i uređajima, rad sa mrežom, podrška za sistem datoteka, tipovi sistema datoteka, kod za upravljanje fizičkom arhitekturom (drajveri)
- Fizička arhitektura (CPU, RAM, U/I uređaji, mrežne kartice, hard diskovi)

### 38. Virtualni sistemi datoteka?

- Virtualni sistemi datoteka omogućavaju pristup korisničkom prostoru ka sistemskim i kernel informacijama.
- Dozvoljavaju aplikacijama da pristupe direktorijumima i datotekama koje ne postoje realno u memoriji, na disku. Oni su napravljeni i kernel ih svežava u toku rada
- Dva najvažnija virtualna sistema datoteka su:
  - proc, najčešće mauntovan na /proc(informacije vezane za operativni sistem(procesi, memorija)
  - sysfs, najčešće mauntovan na /sys(prikaz sistema kao seta uređaja i magistrala)
- postavka /proc
  - sudo mount -t proc none /proc
- postavka /sys
  - sudo mount -t sysfs non /sys
- sysfs i proc tip fajl sistema
- none uređaj ili slika fajl sistema
- /proc i /sys tačke postavljanja

### 39.Šta su i koje su prednosti, a koje mane Drajvera uređaja korisničkog prostora?

- Moguće je implementirati drajvere samo u korisničkom prostoru
- takvim drajverima treba pristup uređajima kroz minimalne generičke kernel drajvere (drajver za štampač i skener)
- prednosti:
  - nema potrebe za znanjem kodiranja u kernelu
  - Drajveri mogu biti napisani u bilo kom jeziku
  - mogu biti vlasnički
  - kod može biti prekinut i debugovan, ne može da stopira kernel
  - može da koristi aritmetiku pokretnog zareza
  - manje kernel kompleksnosti
- Mane:
  - komplikovanije rukovanje prekidima
  - povećano kašnjenje u odnosu na kernel kod



## X RAZVOJ RUKOVOVALACA

### 40. Kako se koriste komande patch?

- patch komanda primenjuje promene na datoteke u trenutnom direktorijumu
  - pravi izmene na postojeće datoteke
  - kreira ili uklanja datoteke i direktorijume
- primeri upotrebe patch komande
  - patch -p<n> <diff\_file>
  - cat diff\_file | patch -p <n>
- n je broj nivoa direktorijuma koji se preskaču u putanjama
- može da se vrati patch sa -R opcijom
- zakrpa može da se testira sa -dry-run opcijom

### 41. Konfiguracija kernela?

- kernel sadrži hiljade drajvera za uređaje, drajvere za fajl sisteme,...
- dostupne su hiljade opcija koje se koriste da se odabirom prevedu delovi izvornog koda kernela
- to je proces definisanja seta opcija kojima želimo da prevedemo naš kernel
- opcije zavise od fizičke arhitekture i mogućnosti koje želimo da kernel ima
- Konfiguracija definiše šta želimo da uključimo u kernel
- čuva se u .config datoteci u root-u kernel koda
- make xconfig/gconfig/menueconfig/oldconfig

### 41. Kako funkcioniše make xconfig?

- to je najčešća grafička sprega koja se koristi za konfiguraciju kernela
- postoji polje za pretragu i dozvoljava da se uključe ili isključe pronađeni parametri
- postoje zavisnosti između opcija u kernelu
- dva tipa zavisnosti:
  - depends on (opcija A koja zavisi od opcije B nije vidljiva dok opcija B nije uključena)
  - select (opcija a zavisi od opcije B, kada se uključi opcija a, automatski se uključi i opcija B)
- u make xconfig se vide sve opcije ,čak i one koje ne mogu da se vide zbog neispunjenih zavisnosti

### 42. Postupak prevođenja kernela u koracima?

- make xconfig
- make
- sudo make install
- sudo make modules\_install

### 43. Navedi neke karakterne i blok datoteke uređaja?

- Karakterne:
  - mogu biti identifikovani po njihovom c tipu (ls -l)
  - tastature, miševi, IrDA, Bluetooth prolaz, konzole, terminali, zvuk, video
  - /dev/ttS0,/dev/pts/1,/dev/input/mouse0,/dev/null
- Blok:

- blok uređaji mogu biti identifikovani sa b tipom (ls -l)
- hda1, fd0, loop0, ram1, sda1...
- hard diskovi, flopi diskovi, ram diskovi, loop uređaji

#### **44. Šta su major i minor brojevi uređaja i kako se kreiraju datoteke uređaja?**

- Major i minor brojeve koristi kernel da veže drajver za datoteku uređaja. Imena datoteka uređaja ništa ne znače kernelu.
- brw-rw----1 root disk 3, 1 Feb 23 2004 hda1 (broj 3 je Major broj, broj 1 je minor broj)
- datoteke se kreiraju : sudo mknod /dev/<device> [c|b] <major> <minor>
- primer sudo mknod /dev/ttyS0 c 4 64

#### **45. Koji su zadatci bootloadera?**

- Izvršava ga fizička arhitektura na podešenoj lokaciji
- inicijalizuje podršku za uređaj gde se nalazi slika kernel
- učitava sliku kernel u RAM
- izvršava sliku kernel

#### **46. Mane init ram diska?**

- potreban je ramdisk i drajver sistema datoteka
- podešena veličina ne mogu da je povećavaju (ZAUZIMA SLOBODAN PROSTOR)
- mora da bude kreiran i menan kao blok uređaj (POTREBAN ROOT)
- kao i u svakom blok uređaju datoteke se prvo čitaju iz skladišta pa se tek onda kopiraju u keš (SPORO)

#### **47. Zašto se koristi initramfs i initrd?**

- initramfs – kernel slika kreira korenski system datoteka, lakše kreiranje( u vreme izgradnje kernela), u poređenju sa initramdiskovima rukuje se samo jednom datotekaom u boot loader uvek je prisutan u kernelu 2.6, to je kompresovana cpio arhiva
- initrd koristimo za kreiranje korenskog sistema datoteka, učitava sliku init ramdiska postavlja korenski system datoteka određen za root kernel parametrom

#### **48. Parametri komandne linije kernela?**

- oni su deo bootloader ovih konfiguracionih podešavanja
- korisni su za izmene ponašanja kernel u vreme pokretanja, bez potrebe za novim prevođenjem
- korisno je za izvođenje naprednih inicijalizacija kernela i drajvera, a bez potrebe za kompleksnim skriptama korisničkog prostora

#### **49. Korisnost rootfs-a na NFS-u?**

- kada radi mreža, korenski system datoteka može biti direktorijum na mašini za razvoj, izvedena od strane NFS-a
- Vrlo je korisno za razvoj Sistema:
  - jednostavno je za ažuriranje datoteka (naročito drajver modula) na korenskom sistemu datoteka bez ponovnog pokretanja
  - može se imati veliki korenski system datoteka čak iako ne postoji podrška za dodatno skladište podataka

### 50. Prvi program korisničkog prostora?

- specificira ga parameter komandne linije /init
- izvršava se na kraju pokretanja kernel
- brine se o startovanju svih ostalih programa korisničkog prostora
- dobija broj porocesa (pid) 1

### 51. Objasniti unakrsno prevođenje kernela?

- kada se prevodi Linux kernel za drugu CPU arhitekturu:
  - mnogo je brže nego nativno prevođenje, kad je ciljani system mnogo sporiji od GNU/Linux radne stanice
  - mnogo je lakše jer je alate za razvoj mnogo lakše za naći na radnoj stanici
  - da bi se napravila razlika sa nativnim prevodiocem, imaju prefix poi menu ciljanog sistema, arhitekture I ponekad biblioteke

### 52. Specificiranje unakrsnog izvršioca?

- CPU arhitektura I prefix unakrsnog izvršioca su definisani u ARCH I CROSS\_COMPILE promenljive u vrhovnom Makefile-u
- **Najlakše rešenje** je da se modifikuje Makefile, odnosno da se u samom Makefilu upisu vrednosti ARCH I CROSS\_COMPILE
- **Drugo rešenje** je da se ARCH I CROSS\_COMPILE postave kroz make komandnu liniju
  - bilo koja promenljiva postavljena kroz make komadnu liniju naglašava bilo koje drugo podešavanje u Makefilu-u
  - make ARCH=sh CROSS\_COMPILE=sh-linux-xconfig
  - mana je to što ne smeju nikad da se zaborave ova podešavanja kad se pokreće make
- **Treće rešenje** postaviti ARCH I CROSS\_COMPILE kao promenljive okruženja u vašem terminalu
  - export ARCH = arm ; export CROSS\_COMPILE= arm-linux-

### 53. Device tree?

- zavisno od arhitekture iskorišteni hardver je ili opisan C kodom direktno unutar kernel ili korišćenjem posebnog jezika za opis hardvera u device TREE
- Device tree source se prevodi u binarnu datoteku Device Tree blob
- bootloader mora pre pokretanja kernel da učita u memoriju I sliku kernel I Device Tree blob

### 54. Šta je devfs, rešenje I ograničenja?

- pokazuje samo prisutne uređaje
- koristi različita imena u /dev, što dovodi do problema u skriptama
- nema fleksibilnost u imenovanju uređaja
- ne dozvoljava dinamičku alokaciju major I minor brojeva
- Zahteva čuvanje načina imenovanja uređaja u kernel memoriji

### 57. Za šta se koristi mdev?

- Da bi koristili mdev, proc I sysfs sistemi datoteka moraju biti postavljeni
- mdev mora biti uključen kao hotplug upravljač

- Potrebno je reći mdev-u da kreira /dev unose odgovarajuće detektovanim uređajima tokom pokretanja, dok mdev još uvek nije radio mdev —s
- Ponašanje je naznačeno u /etc/mdev.conf sa formatom <device regex> <uid> <octal permissions>

## 58. Obezbeđivanje alata za prevođenje?

- **Samostalno pravljenje alata** za unakrsno prevođenje
  - težak i naporan zadatak
  - može trajati danima ili nedeljama
  - Treba naučiti dosta detalja, mnogo odluka
  - potrebna su kernel zaglavlja i izvorni kod C biblioteka
  - potrebno je poznavanje trenutnih gcc problema i zakrpa
- **Preuzimanje prebedenog alata** za prevođenje
  - prednost: najjednostavnije i najzgodnije rešenje
  - mane: ne postoji mogućnost za fina podešavanja alata prema potrebama
  - treba se uveriti da pronađeni alat odgovara zahtevima

## 59. Pravljenje korenskog sistema datoteka pomoću buildroot-a ?

- podržava većinu arhitektura
- automatski dobavlja izvorne kodove, primenjuje zakrpe i instalira programe koji su mu potrebni
- pored korisničkih programa može da prevede većinu standardnih, često korišćenih programa
- veoma lak za korišćenje (make menuconfig, make)

## 60. Emulatori?

- **Emulatorski sistemi** su korisni za isprobavanje čitavih sistema, uključujući i kernel
- **Korisnički emulatori** korisni za pokretanje i debugovanje korisničkih programa namenjenih drugim arhitekturama

## 61. BusyBox?

- Većina komandi Unix komandne linije u jednoj izvršnoj datoteci
- Uključuje i web server
- Lako se konfiguriše
- najbolji izbor za initramfs/initrd sa kompleksnim skriptama, male i srednje ugrađene sisteme

## 62. Konfiguracija BusyBox?

- Konfiguracija BusyBox-a (stvara .config datoteku)
  - make defconfig (dobra opcija za početak korišćenja BusyBox-a, konfiguriše BusyBox sa svim opcijama za običnog korisnika)
  - make allconfig (isključuje sve opcije, koristi se kad se traži minimalni skup)
  - make xconfig (grafički) ili make menuconfig (tekstualni) (to se koristi i kod linux kernela)

## 63. Prevođenje BusyBox-a?

- Ukoliko se prevodi za drugu platform, potrebno je izabrati odgovarajuću arhitekturu ciljne platforme  
ARCH = arm, CROSS\_COMPILE = arm-linux-
- Dodati putanju do alata za prevođenje u varijablu okruženja PATH (export PATH...)

- Prevođenje BusyBox-a (make)
- Instalacija (stvara strukturu direktorijuma u \_install sa simboličkim linkovima na buszbox izvršnu datoteku) (make install)

#### **64. Pravljenje novog BusyBox apleta?**

- Korisno za pravljenje sopstvenih malih izvršnih datoteka
- Mogu da se krosite sve prednosti koje donosi BusyBox
- Dobijaju se manje datoteke nego kada bismo pravili izolovanu izvršnu datoteku
- koriste se i prednosti koje pruža BusyBox veyane za konfiguraciju i prevođenje za druge platform
- licenca aplikacije će morati da bude GPL

#### **65. Priprema slike kernel za U-boot?**

- U-boot mora da ima dodatne informacije u slici kernel ili initrd-a
- mkimage- alat za pripremu slike kernel koji se nalazu u sklopu U boot izvornog koda
- pravljenje odgovarajuće slike kernel

#### **66. Kako se konfiguriše tftp?**

- Česta pojava u toku razvoja je preuzimanje slike kernel **preko mreže**
- Instrukcije za xinetd bazirane sisteme
- Instalirati tftp- server paket ako je potrebno
- Ukloniti disable = yes u datoteci /etc/xinetd.d/tftp
- Kopirati datoteke u tftpboot/ direktorijum
- Može se desiti da mora da se isključi SELinux u /etc/selinux/config datoteci
- Restartovati xinetd

- Instrukcije za linux sisteme bazirane na Debian u
- Instalirati tftp-hpa paket ako je potrebno
- postaviti RUN\_DEAMON= 'yes' u datoteci /etc/default/tftpd-hpa
- kopirati datoteke u direktorijum /var/lib/tftpboot
- restartovati poslužioca /etc/init.d/tftpd-hpa restart

#### **67.**